Lab 2 - LIKE a covering index

Skripty su v `2/sql/` , vystupy budu v `2/outputs/` .

Plan

Lab 2 riesi tieto veci:

- `LIKE '%5733'` je pomale, lebo hladame podla konca retazca.
- Bezne B-tree indexovanie nepomoze pri pattern-e, ktory zacina `%` .
- `LIKE '57%'` vie index pouzit, lebo hladame podla zaciatku retazca.
- Suffix search sa da zrychlit cez `reverse(supplier_ico)` a functional index.
- Covering index `(department, customer)` moze dovolit `Index Only Scan` .

Skripty

- `00_inspect.sql` - zakladne pocty pre pouzite dotazy
- `01_suffix_like_no_index.sql` - `supplier_ico LIKE '%5733'` bez indexu
- `02_suffix_like_with_index.sql` - rovnaky dotaz s indexom na `supplier_ico`
- `03_prefix_like_no_index.sql` - `supplier_ico LIKE '57%'` bez indexu
- `04_prefix_like_with_index.sql` - rovnaky dotaz s indexom
- `05_suffix_like_reverse_index.sql` - suffix search cez `reverse()`
- `06_department_no_index.sql` - `department` dotaz bez indexu
- `07_department_index.sql` - index na `department`
- `08_department_customer_covering_index.sql` - covering index `(department, customer)`

00 - Inspect

Tabulka `documents` ma 351224 riadkov.

Pouzite dotazy vracaju:

Dotaz	Pocet riadkov
`supplier_ico LIKE '%5733'`	5

Dotaz	Pocet riadkov
`supplier_ico LIKE '57%'`	40
`department = 'Rozhlas a televizia Slovenska'`	16917

01 a 02 - Suffix LIKE

Dotaz:

```sql
SELECT *
FROM documents
WHERE supplier_ico LIKE '%5733';
```

Experiment	Index	Plan	Total runtime
01	bez indexu	`Seq Scan`	65.200 ms
02	`supplier_ico text_pattern_ops`	`Seq Scan`	63.793 ms

Index nepomohol, lebo pattern zacina znakom `%`. B-tree index vie efektivne hladat od zaciatku hodnoty, ale tu hladame podla konca retazca.

03 a 04 - Prefix LIKE

Dotaz:

```sql
SELECT *
FROM documents
WHERE supplier_ico LIKE '57%';
```

Experiment	Index	Plan	Total runtime
03	bez indexu	`Seq Scan`	64.968 ms
04	`supplier_ico text_pattern_ops`	`Bitmap Index Scan` + `Bitmap Heap Scan`	0.576 ms

Tu index pomohol, lebo pattern ma pevny zaciatok `57`. Planner ho vie prepisat na range podmienku nad indexom:

```
supplier_ico >= '57'
AND supplier_ico < '58'
```

Vo vystupe je to zapisane ako `Index Cond` s operatormi `~>=~` a `~<~`.

05 - Rychly suffix search cez `reverse()`

Dotaz:

```
SELECT *
FROM documents
WHERE reverse(supplier_ico) LIKE reverse('5733') || '%';
```

Index:

```
CREATE INDEX index_documents_on_reverse_supplier_ico
ON documents (reverse(supplier_ico) text_pattern_ops);
```

Plan bol `Index Scan` a `Total runtime` bol 0.073 ms.

Tymto sa suffix search zmeni na prefix search. Hodnota `5733` sa otoci na `3375`, preto databaza hlada `reverse(supplier_ico) LIKE '3375%'`, co uz B-tree index pouzit vie.

06 az 08 - Covering index

Dotaz:

```
SELECT customer
FROM documents
WHERE department = 'Rozhlas a televizia Slovenska';
```

Experiment	Index	Plan	Total runtime
06	bez indexu	`Seq Scan`	51.625 ms

Experiment	Index	Plan	Total runtime
07	`department`	`Bitmap Index Scan` + `Bitmap Heap Scan`	26.232 ms
08	`(department, customer)`	`Index Only Scan`	3.132 ms

Index na `department` pomohol najst riadky rychlejsie, ale databaza este musela citat hodnotu `customer` z tabulky. Covering index `(department, customer)` obsahuje aj stlpec, ktory vraciame, preto PostgreSQL pouzil `Index Only Scan`. Vo vystupe je `Heap Fetches: 0`, cize riadky nemusel docitavat z tabulky.

Zaver

Lab 2 ukazal tri prakticke veci:

- `LIKE '%suffix'` je pomale a obycajny B-tree index mu nepomoze.
- `LIKE 'prefix%'` vie B-tree index s `text_pattern_ops` vyuzit.
- Covering index moze byt rychlejsi, ked dotaz potrebuje iba stlpce, ktore su priamo v indexe.